All-Depth Target-One Barrier Exclusion for the Negative Collatz Shadow

A rigorous audit of the theorem, its certificates, the current proof architectures, and the remaining synchronization problem

Principia Math Collatz Project
Technical audit prepared for Anton Shakov

17 August 2026

Status. The principal result of this document is an all-depth, computer-assisted theorem about a constrained language for the accelerated $3x-1$ map. The proof has an analytic part and a finite exact certificate. It is *not* a proof of the Collatz conjecture. The remaining debts are identified precisely in section 11.

Abstract

For a positive valuation word $a=(a_0,\dots,a_{r-1})$, define $A_k=\sum_{i<k}a_i$ and $C_0=0$, $C_{k+1}=3C_k+2^{A_k}$. We prove that

$$3^r \mid C_r+2^{A_r}, \qquad 2^{A_k}<3^k \quad (1\le k\le r)$$

forces $a_0=\cdots=a_{r-1}=1$. Equivalently, the all-ones words are the only target-1 terminal words that remain strictly on the expanding side of the coefficient barrier at every prefix for the accelerated negative map $U_-(m)=(3m-1)/2^{v_2(3m-1)}$.

The proof has four ingredients. Terminal divisibility reconstructs an exact positive $3x-1$ trajectory ending at 1. Equal-time translation on parity cylinders gives a sharp shell-injection bound for every finite simple negative shortcut trajectory. This yields the universal reciprocal-mass estimate

$$\sum_i \frac{1}{3x_i-1}<10.789.$$

The product formula along a barrier trajectory then bounds its initial seed by 3^{11} times its terminal target. A complete scan of the odd seeds below 200000, implemented independently in C++ and Python, leaves only the seed 1. The numerical shell sum through level 1000, its analytic tail, every source file, every retained output, and cryptographic hashes are specified.

We also audit the surrounding Collatz project. We separate proved affine, locality, coalescence, inverse-fiber, and record-compression theorems from finite evidence, conditional completion criteria, and refuted or insufficient routes. The target-1 terminal debt is closed; nonperiodic synchronization and positive-cycle exclusion are not. The final appendices give Lean-oriented proof skeletons for the all-depth theorem and for three possible routes from the present frontier to the Collatz conjecture.

Contents

1 Scope, notation, and exact claim **4**

2 Position relative to the literature and provenance **5**

3 Affine word calculus and terminal reconstruction **5**

3.1 Positive and negative affine identities 5

3.2 Terminal suffix locality 6

3.3 Terminal divisibility reconstructs an exact trajectory 6

4 Equal-time parity cylinders **7**

5 Finite-orbit shell sparsity **8**

6 A universal reciprocal-mass constant **9**

6.1 The analytic tail 9

6.2 The finite shell certificate 10

7 Fixed-target compression and proof of the main theorem **11**

8 Certificate specification and reproducibility **12**

8.1 Retained files and commands 12

8.2 Exact retained outputs 13

8.3 SHA–256 manifest 14

8.4 Trust boundary 14

9 What the theorem represents and what it does not **14**

9.1 A local language theorem of genuinely infinite depth 14

9.2 Consequence for positive Collatz suffixes 15

9.3 Why positive cycles remain separate 15

10 Audit of the rigorous project ledger **16**

10.1 Established exact identities and reductions 16

10.2 Divergent-orbit compression 17

10.3 Exact rewrite macrosemantics 18

10.4 Finite certificates retained as evidence but not used in theorem 1.2 18

10.5 Exact no-go results and routes that were not completed 19

11 The current proof frontier **19**

11.1 Architecture A: terminal exclusion plus synchronization plus cycle exclusion 20

11.2 Architecture B: stationary inverse-fiber extinction 20
11.3 Architecture C: first crossing and high-surplus carry innovation 21
11.4 A practical hierarchy of the remaining tasks 21

12 Discussion and assessment **21**
12.1 Why this is genuine progress 21
12.2 Why it is not yet close enough to declare Collatz solved 22
12.3 Most promising exact targets 22

A Dependency graph for the all-depth theorem **22**

B Pseudocode for the exact certificates **23**
B.1 Mass certificate 23
B.2 Accelerated seed certificate 23

C Lean-oriented proof and completion skeletons **24**

1 Scope, notation, and exact claim

The classical Collatz map is

$$T(n) = \begin{cases} n/2, & 2 \mid n, \\ 3n+1, & 2 \nmid n. \end{cases}$$

It is convenient to work on odd positive integers with the accelerated map

$$U_+(n) = \frac{3n+1}{2^{v_2(3n+1)}}.$$

The auxiliary negative map is

$$U_-(m) = \frac{3m-1}{2^{v_2(3m-1)}}.$$

The negative map is not a replacement for the positive Collatz dynamics. It is a dual arithmetic shadow whose finite valuation words carry the same powers of 2 and 3, with the affine carry entering with the opposite sign.

Definition 1.1 (Valuation data)**.** Let $a = (a_0, \dots, a_{r-1}) \in \mathbb{N}_{\geq 1}^r$. Put

$$A_0 = 0, \qquad A_k = \sum_{i=0}^{k-1} a_i,$$

and define

$$C_0 = 0, \qquad C_{k+1} = 3C_k + 2^{A_k}.$$

Thus

$$C_k = \sum_{i=0}^{k-1} 3^{k-1-i} 2^{A_i}.$$

The word is called *barrier-admissible* when

$$2^{A_k} < 3^k \qquad (1 \leq k \leq r), \tag{1.1}$$

and *target-y terminal*, for an odd positive integer y, when

$$3^r \mid C_r + 2^{A_r} y. \tag{1.2}$$

The principal theorem is the following.

Theorem 1.2 (All-depth target-1 barrier exclusion)**.** *Let $a = (a_0, \dots, a_{r-1}) \in \mathbb{N}_{\geq 1}^r$. If*

$$3^r \mid C_r + 2^{A_r} \quad \textit{and} \quad 2^{A_k} < 3^k \quad (1 \leq k \leq r),$$

then

$$a_0 = a_1 = \cdots = a_{r-1} = 1.$$

The qualifier "all-depth" refers to the word length r: no upper bound on r occurs in the statement. The theorem is nevertheless computer-assisted, because its analytic argument compresses every possible length to a finite seed range and an exact finite scan settles that range.

2 Position relative to the literature and provenance

The strongest unconditional global results remain far from pointwise totality. Tao proved that the minimum of the Collatz orbit is below every function tending to infinity for logarithmic-density almost all starting values [11]. Barina's current published computation verifies convergence for every starting value below 2^{71} [1]. Krasikov and Lagarias proved a power-law lower bound for the number of starting values known to reach 1 [7]. For cycles, Simons and de Weger developed strong Diophantine and computational bounds on the number of local minima [10], and Hercher proved that a nontrivial positive Collatz cycle would have at least 92 local minima [5]. This is the strongest peer-reviewed local-minimum bound located in the present audit. A June 2026 independent preprint claims an exact computer-assisted extension excluding m-cycles with $m \leq 95$, conditional on the Barina verification bound and the Simons–de Weger inputs [12]; that certificate has not been independently audited here and is not used below.

The present theorem has a different scope. It classifies a constrained language in the negative $3x-1$ shadow. It neither verifies additional ordinary positive seeds nor excludes a positive Collatz cycle. In particular, it is not comparable by implication with the computational bound 2^{71} or Tao's almost-all theorem.

The ingredients have recognizable ancestors.

(i) Affine word formulae, parity cylinders, and 2-adic symbolic coding are part of the classical $3x+1$ literature; see Lagarias' survey [8], Wirsching's monograph [13], and the Bernstein–Lagarias conjugacy [2].
(ii) Orbit sparsity and reciprocal-sum arguments are closely related to the work of Garcia and Tal on generalized $3n+1$ orbits [4].
(iii) The mixed binary–ternary rewrite-system viewpoint comes from Yolcu, Aaronson, and Heule [14].
(iv) The target-shadow formulation, terminal suffix locality, coalescing minima, and the equal-time shell injection were developed in the present Principia Math Collatz project. The project direction and work ledger were maintained by Anton Shakov; derivations, audits, and executable certificates were developed with GPT-assisted mathematical exploration and then rechecked symbolically and computationally for this report.

A targeted search of the standard surveys, the generalized Syracuse literature, the rewrite-system literature, and recent work on paradoxical or extremal words [9, 3, 6] did not locate a published theorem equivalent to theorem 1.2. This is a search report, not a priority claim. A formal priority claim would require a broader expert literature review and external peer verification.

3 Affine word calculus and terminal reconstruction

3.1 Positive and negative affine identities

Proposition 3.1 (Affine identities)**.** *Suppose $a = (a_0, \dots, a_{r-1})$ is the exact valuation word of a positive accelerated trajectory $n_0, \dots, n_r$ under U_+. Then*

$$2^{A_k} n_k = 3^k n_0 + C_k \qquad (0 \leq k \leq r). \tag{3.1}$$

If it is the exact valuation word of a positive accelerated trajectory $m_0, \dots, m_r$ under U_-, then

$$2^{A_k} m_k = 3^k m_0 - C_k \qquad (0 \leq k \leq r), \tag{3.2}$$

or equivalently

$$3^k m_0 = 2^{A_k} m_k + C_k. \tag{3.3}$$

Proof. For the positive map, $2^{a_k} n_{k+1} = 3n_k + 1$. Assuming (3.1) at time k and multiplying this recurrence by 2^{A_k} gives

$$2^{A_{k+1}} n_{k+1} = 3(3^k n_0 + C_k) + 2^{A_k} = 3^{k+1} n_0 + C_{k+1}.$$

The negative identity follows identically from $2^{a_k} m_{k+1} = 3m_k - 1$. □

If words u and v are concatenated, their carries satisfy

$$C(uv) = 3^{|v|} C(u) + 2^{A(u)} C(v). \tag{3.4}$$

This follows either from the defining sum or by composing the affine maps.

3.2 Terminal suffix locality

Proposition 3.2 (Full terminal residue locality)**.** *Let a positive U_+ trajectory have endpoint X, and let $v = (a_{j-r}, \dots, a_{j-1})$ be its last r valuation exponents. Put $B = A(v)$ and $C_v = C(v)$. Then, for every integer c,*

$$X \equiv c \pmod{3^r} \quad \Longleftrightarrow \quad C_v - 2^B c \equiv 0 \pmod{3^r}. \tag{3.5}$$

In particular,

$$3^r \mid X + 1 \quad \Longleftrightarrow \quad 3^r \mid C_v + 2^B. \tag{3.6}$$

Proof. If Y is the positive odd state at the start of the suffix, then

$$2^B X = 3^r Y + C_v.$$

Reduction modulo 3^r and invertibility of 2^B modulo 3^r prove the claim. □

3.3 Terminal divisibility reconstructs an exact trajectory

Theorem 3.3 (Targeted terminal duality)**.** *Let $a = (a_0, \dots, a_{r-1}) \in \mathbb{N}_{\geq 1}^r$ and let y be a positive odd integer. The following are equivalent.*

(i) $3^r \mid C_r + 2^{A_r} y$.

(ii) *There are positive odd integers $m_0, \dots, m_r = y$ satisfying*

$$3m_i - 1 = 2^{a_i} m_{i+1} \qquad (0 \leq i < r). \tag{3.7}$$

Every exponent in (3.7) *is then the exact 2-adic valuation of $3m_i - 1$.*

Proof. For $0 \leq i \leq r$, let D_i denote

$$D_i = C(a_i, \dots, a_{r-1}) + 2^{a_i + \cdots + a_{r-1}} y,$$

with $D_r = y$. Splitting off the first letter of a suffix gives

$$D_i = 3^{r-i-1} + 2^{a_i} D_{i+1}. \tag{3.8}$$

Assume $3^{r-i} \mid D_i$. Reducing (3.8) modulo 3^{r-i-1} shows that $3^{r-i-1} \mid D_{i+1}$. Starting from (i), this propagates divisibility to every suffix. Define

$$m_i = \frac{D_i}{3^{r-i}}.$$

Then (3.8) becomes

$$3m_i = 1 + 2^{a_i} m_{i+1}.$$

Backward induction from the positive odd integer $m_r = y$ shows that each m_i is a positive odd integer: the numerator $1 + 2^{a_i} m_{i+1}$ is odd, and divisibility by 3 has already been proved. Consequently $3m_i - 1 = 2^{a_i} m_{i+1}$ with m_{i+1} odd, so the valuation is exactly a_i. Conversely, iterating (3.7) gives $3^r m_0 = C_r + 2^{A_r} y$. □

This theorem is the first decisive reduction: a formal congruence on a word is not merely necessary. It produces an ordinary positive integer trajectory in the negative map.

4 Equal-time parity cylinders

Define the negative shortcut map on nonnegative integers by

$$S_-(x) = \begin{cases} x/2, & x \equiv 0 \pmod 2, \\ (3x-1)/2, & x \equiv 1 \pmod 2. \end{cases} \tag{4.1}$$

For $k \geq 0$, let $s_k(x)$ be the number of odd inputs among $x, S_-(x), \ldots, S_-^{k-1}(x)$.

Lemma 4.1 (Parity cylinder)**.** *Congruence modulo 2^k determines the first k parity bits of a shortcut trajectory. Moreover, every binary word of length k occurs for exactly one residue class modulo 2^k.*

Proof. If $x \equiv y \pmod{2^k}$, then x and y have the same parity. After one common branch, their images are congruent modulo 2^{k-1}. Induction gives the first assertion.

For injectivity, suppose two residues have the same parity word, of weight o. Following the common affine branch word gives

$$S_-^k(x) - S_-^k(y) = \frac{3^o}{2^k}(x - y).$$

The left side is an integer and 3° is odd, so $2^k \mid x - y$. The two residues are equal. There are 2^k residues and 2^k binary words, hence the map is bijective. □

Lemma 4.2 (Equal-time translation)**.** *For nonnegative integers x, q, k,*

$$S_-^k(x + q2^k) = S_-^k(x) + q3^{s_k(x)}. \tag{4.2}$$

Proof. The two initial values are congruent modulo 2^k, so by theorem 4.1 they follow the same first k branches. Their initial difference is $q2^k$. Each step divides the difference by 2, and each odd branch additionally multiplies it by 3. After k steps the difference is $q3^{s_k(x)}$. □

For a fixed parity word of length k and weight o, the corresponding affine formula has the form

$$2^k S_-^k(u) = 3^o u - D, \qquad D \geq 0. \tag{4.3}$$

Therefore, whenever $0 \leq u < 2^k$,

$$0 \leq S_-^k(u) < 3^o. \tag{4.4}$$

The nonnegativity follows because the shortcut map preserves $\mathbb{N} \cup \{0\}$; the strict upper bound follows from $u < 2^k$ and $D \geq 0$.

5 Finite-orbit shell sparsity

Let $x_0, x_1, \ldots, x_L$ be a positive S_- trajectory with no repeated state. For $k \geq 0$, put

$$N_k = \#\{i : 2^k \leq x_i < 2^{k+1}\}. \tag{5.1}$$

Define

$$B_k = \min_{0 \leq t \leq k} \left(\sum_{o=t+1}^{k} \binom{k}{o} + \sum_{o=0}^{t} 3^o \right). \tag{5.2}$$

Theorem 5.1 (Shell sparsity)**.** *For every $k \geq 0$,*

$$N_k \leq \min\{2^k, B_k + k\}. \tag{5.3}$$

Proof. The bound $N_k \leq 2^k$ is immediate because the shell $[2^k, 2^{k+1})$ contains exactly 2^k integers.

Fix $t \in \{0, \ldots, k\}$. Temporarily discard the final k positions of the trajectory. Every remaining shell point has a unique representation

$$x_i = 2^k + u_i, \qquad 0 \leq u_i < 2^k. \tag{5.4}$$

By theorem 4.1, exactly $\binom{k}{o}$ residues u have a first-k parity word of weight o. Hence the number of retained shell points whose weight exceeds t is at most

$$\sum_{o=t+1}^{k} \binom{k}{o}. \tag{5.5}$$

Now fix $o \leq t$. For a retained shell point of weight o, define its *base endpoint* to be $S_-^k(u_i)$. By (4.4), the base endpoint is one of the 3° integers in $[0, 3^o)$. We claim that, for fixed o, no two retained shell points have the same base endpoint. Indeed, if $x_i = 2^k + u_i$ and $x_j = 2^k + u_j$ had the same weight and the same base endpoint, theorem 4.2 with $q = 1$ would give

$$x_{i+k} = S_-^k(u_i) + 3^o = S_-^k(u_j) + 3^o = x_{j+k}.$$

Both future positions exist because the last k positions were discarded. This contradicts nonrepetition. Thus the number of retained points of weight o is at most 3°. Summing over $o \leq t$, adding (5.5), and restoring at most k discarded positions gives

$$N_k \leq \sum_{o=t+1}^{k} \binom{k}{o} + \sum_{o=0}^{t} 3^o + k.$$

Minimizing over t proves (5.3). □

The role of equal time is essential. It turns a collision in a finite set of base endpoints into an actual repeated trajectory state. Counting parity words alone would not provide this injection.

6 A universal reciprocal-mass constant

Every point in the kth shell contributes at most $(3 \cdot 2^k - 1)^{-1}$. Therefore theorem 5.1 gives

$$\sum_{i=0}^{L} \frac{1}{3x_i - 1} \le \sum_{k=0}^{\infty} \frac{\min\{2^k, B_k + k\}}{3 \cdot 2^k - 1}. \tag{6.1}$$

We now prove that the right side is below 10.789.

6.1 The analytic tail

Lemma 6.1 (Chernoff form for the binomial tail)**.** *Let $1/2 < p < 1$ and let $t = \lfloor pk \rfloor$. Then*

$$\sum_{o=t+1}^{k} \binom{k}{o} \le \left(p^{-p} (1-p)^{-(1-p)} \right)^k. \tag{6.2}$$

Proof. For $z > 1$ and every $o > pk$, one has $z^o > z^{pk}$. Hence

$$\sum_{o > pk} \binom{k}{o} \le z^{-pk} \sum_{o=0}^{k} \binom{k}{o} z^o = z^{-pk} (1+z)^k.$$

The right side is minimized at $z = p/(1-p)$, which yields (6.2). □

Take $p = 61/100$ and $t = \lfloor 61k/100 \rfloor$. The exact integer inequalities

$$3^{61} < 2^{97}, \tag{6.3}$$

$$100^{100} < 2^{97} 61^{61} 39^{39} \tag{6.4}$$

are equivalent to

$$3^{61/100} < 2^{97/100}, \qquad p^{-p} (1-p)^{-(1-p)} < 2^{97/100}.$$

By theorem 6.1,

$$\sum_{o=t+1}^{k} \binom{k}{o} < 2^{97k/100}. \tag{6.5}$$

Also,

$$\sum_{o=0}^{t} 3^o = \frac{3^{t+1} - 1}{2} < \frac{3}{2} 3^t < \frac{3}{2} 2^{97k/100}. \tag{6.6}$$

Consequently

$$B_k < \frac{5}{2} 2^{97k/100}. \tag{6.7}$$

For $k \ge 1001$,

$$\frac{B_k + k}{3 \cdot 2^k - 1} < \frac{5}{4} 2^{-3k/100} + \frac{k}{2^{k+1}}. \tag{6.8}$$

Grouping the first series into blocks of 100 terms gives

$$\begin{aligned}\frac{5}{4}\sum_{k=1001}^{\infty} 2^{-3k/100} &< \frac{5}{4}\sum_{j=0}^{\infty} 100\, 2^{-30-3j} \\ &= \frac{1000}{7\cdot 2^{30}}.\end{aligned} \tag{6.9}$$

The standard differentiated geometric series gives

$$\sum_{k=1001}^{\infty} \frac{k}{2^{k+1}} = \frac{1002}{2^{1001}}. \tag{6.10}$$

Thus the exact tail bound is

$$\sum_{k=1001}^{\infty} \frac{\min\{2^k, B_k + k\}}{3\cdot 2^k - 1} < \frac{1000}{7\cdot 2^{30}} + \frac{1002}{2^{1001}} < 1.331 \times 10^{-7}. \tag{6.11}$$

6.2 The finite shell certificate

For $0 \le k \le 1000$, define

$$n_k = \min\{2^k, B_k + k\}, \qquad u_k = \left\lceil 10^{15} \frac{n_k}{3\cdot 2^k - 1} \right\rceil. \tag{6.12}$$

The retained CSV certificate contains (k, B_k, t_k, n_k, u_k) for every k, where t_k is the first minimizing threshold in (5.2). An independent verifier rebuilds Pascal's triangle, recomputes every threshold, and checks every row.

Certificate 6.2 (Finite mass certificate)**.** The exact verified integer identity is

$$\sum_{k=0}^{1000} u_k = 10{,}788{,}901{,}977{,}820{,}654. \tag{6.13}$$

Therefore

$$\sum_{k=0}^{1000} \frac{n_k}{3\cdot 2^k - 1} \le 10.788901977820654. \tag{6.14}$$

Combining (6.11) and (6.14) gives the certified rational upper bound

$$10.788902110866736\ldots < 10.789. \tag{6.15}$$

No floating-point comparison enters the proof. Decimal values in (6.14)–(6.15) are displays of exact rational inequalities. The proof-relevant data are the integers in (6.3), (6.4), (6.13), and the rational tail in (6.11).

Theorem 6.3 (Universal reciprocal mass)**.** *Every finite positive simple trajectory $x_0, \ldots, x_L$ of the negative shortcut map satisfies*

$$\sum_{i=0}^{L} \frac{1}{3x_i - 1} < 10.789. \tag{6.16}$$

Proof. Combine (6.1), theorem 6.2, and (6.11). □

7 Fixed-target compression and proof of the main theorem

Theorem 7.1 (Fixed-target seed compression)**.** *Let*

$$m_0 \longmapsto m_1 \longmapsto \cdots \longmapsto m_r = y$$

be an exact trajectory of U_- on positive odd integers whose expansion to shortcut steps has no repeated state, with valuation word a satisfying

$$2^{A_j} < 3^j \qquad (1 \le j \le r).$$

Then

$$m_0 < e^{10.789} y < 3^{11} y = 177147y. \tag{7.1}$$

Proof. Each accelerated step satisfies

$$\frac{m_{j+1}}{m_j} = \frac{3m_j - 1}{2^{a_j} m_j} = \frac{3}{2^{a_j}} \left(1 - \frac{1}{3m_j}\right).$$

Multiplication gives

$$\frac{y}{m_0} = \frac{3^r}{2^{A_r}} \prod_{j=0}^{r-1} \left(1 - \frac{1}{3m_j}\right). \tag{7.2}$$

The full barrier gives $3^r/2^{A_r} > 1$, hence

$$\log \frac{m_0}{y} < -\sum_{j=0}^{r-1} \log \left(1 - \frac{1}{3m_j}\right). \tag{7.3}$$

For $0 < u < 1$,

$$-\log(1-u) = \int_0^u \frac{\mathrm{d}t}{1-t} \le \frac{u}{1-u}.$$

With $u = 1/(3m_j)$,

$$-\log \left(1 - \frac{1}{3m_j}\right) \le \frac{1}{3m_j - 1}. \tag{7.4}$$

Expand the accelerated trajectory into shortcut steps. Before the terminal state this shortcut trajectory is simple, and the accelerated odd states are a subset of its states. Thus theorem 6.3 gives

$$\log \frac{m_0}{y} < 10.789.$$

Finally $10.789 < 11$ and $e < 3$ (for example, from the exponential series), so $e^{10.789} < 3^{11}$. □

The final finite input is the following exact computation.

Certificate 7.2 (Target-1 seed scan)**.** Every odd seed m_0 with $1 \le m_0 < 200000$ was followed under exact integer arithmetic until the first occurrence of one of the following mutually exclusive events:

(i) the orbit reaches 1;
(ii) a state repeats;

(iii) the strict accelerated barrier fails;
(iv) the explicit step cap is reached; or
(v) a checked fixed-width operation would overflow.

The C++ implementation returned

reaches 1	1
repeated-state cycles	2
barrier failures	99997
step-cap survivors	0
overflows	0.

The unique terminal hit was $(m_0, r) = (1, 0)$. A separate Python implementation on the shortcut map, using arbitrary-precision integers, returned the same partition.

Proof of theorem 1.2. By theorem 3.3, terminal divisibility reconstructs a positive exact trajectory

$$m_0 \longmapsto \cdots \longmapsto m_r = 1$$

under U_-. Let τ be the first index with $m_\tau = 1$. If $\tau > 0$, then the trajectory up to m_τ has no repeated state: a repeat in a deterministic map would trap the orbit in a cycle and prevent a later first hit of 1. The prefix barrier remains valid. Applying theorem 7.1 to this truncated path gives

$$1 < m_0 < 177147 < 200000.$$

This contradicts theorem 7.2, which has no such seed. Hence $\tau = 0$ and $m_0 = 1$.

Now $3 \cdot 1 - 1 = 2$. Since the next state is odd, the exact valuation is $a_0 = 1$ and $m_1 = 1$. Repeating this argument proves $a_i = 1$ and $m_i = 1$ for every $i < r$. □

8 Certificate specification and reproducibility

Only the certificates in this section are logical dependencies of theorem 1.2. Earlier barrier-tree, order, and excursion computations are useful evidence but are not used in the proof.

8.1 Retained files and commands

The companion archive contains:

File	Function
`mass_certificate.py`	Computes every B_k for $k \leq 1000$, the upward-rounded rational sum, and the exact analytic-tail inequalities.
`mass_shells_k0_1000.csv`	The 1001-row finite shell certificate.
`verify_mass_csv.py`	Independent Pascal-row verifier for every CSV field and the total.
`accelerated_seed_certificate.cpp`	Exact accelerated U_- scan using checked `unsigned __int128` multiplication and arbitrary-precision barrier powers.
`shortcut_seed_certificate.py`	Independent shortcut scan using Python arbitrary-precision integers.
`*.out`	Retained standard output for each run.
`ENVIRONMENT.txt`	Python, compiler, and operating-system identifiers.
`SHA256SUMS`	Cryptographic digest of every retained proof artifact.

The reproduction commands are

```
python mass_certificate.py --K 1000 --csv mass_shells_k0_1000.csv
python verify_mass_csv.py
python shortcut_seed_certificate.py --cap 200000 --step-cap 100000
g++ -O2 -std=c++20 -Wall -Wextra -Werror \
  accelerated_seed_certificate.cpp -o accelerated_seed_certificate
./accelerated_seed_certificate 200000 100000
sha256sum -c SHA256SUMS
```

8.2 Exact retained outputs

The mass computation reports

```
scaled_upward_sum=10788901977820654
check_3pow61_lt_2pow97=True
check_entropy_integer_inequality=True
tail_upper_decimal=1.330460820879255e-07
certified_total_upper_decimal=10.788902110866736
certified_total_lt_10_789=True
```

The independent CSV verifier reports

```
rows_verified=1001
scaled_upward_sum=10788901977820654
certificate_passed=True
```

The shortcut and accelerated seed scans report, respectively,

```
status_counts={'barrier_failure': 99997, 'cycle': 2, 'hit_1': 1}
terminal_hits=[(1, 0)]
```

```
longest_resolution_steps=132
longest_resolution_seed=149345
certificate_passed=True
```

```
hit_1=1
cycle=2
barrier_failure=99997
step_cap=0
overflow=0
longest_resolution_steps=83
longest_resolution_seed=149345
terminal_hits= (1,0)
certificate_passed=True
```

The different longest-resolution lengths are expected: one program counts accelerated odd steps, while the other counts every shortcut step.

8.3 SHA–256 manifest

File	SHA–256
mass_certificate.py	8d83628d918c9828207be42beb26633b770722d4561c2db7e4e2afcb0856c7fa
verify_mass_csv.py	76426c4f5400e98fcc31af15d119a54cbd5268d836291b2816429a841082759f
shortcut_seed_certificate.py	615e0badd5b6ce48834b9541ab2e3d010526a5cdba5d24565cea0f284d06a45a
accelerated_seed_certificate.cpp	4b9bfa5bca84fdd064f1fee0cb85e8ce74985fa06c6bb1dee98fe67ec91c4620
mass_shells_k0_1000.csv	3997b941ae24e59e262636d51ab5baa82b6262de0cd04f6a5812978e7ebea266
mass_certificate.out	8630b315ec64a84983041f428f804f48c56be27b3a122ab52ec593bd76acb21f
verify_mass_csv.out	e7ceaab0085fd8f32368b26df80ff05709f203f95608687839f47e246d91c032
shortcut_seed_certificate.out	efb67cb920fd8cc9a89e2a4410c1a8ab47dc6560d96f792709abf8b81fcb56f8
accelerated_seed_certificate.out	30f5dd9f84454be45a7840b6a3a52f0edbfeccb42c4fb8e6cf9fdd42e86a3348
ENVIRONMENT.txt	3df8386c359b194a6414fe70b80e4231b3cfd018f9c02924b2e7c07f356e353c

8.4 Trust boundary

The finite theorem can be checked at several levels.

(i) A reader may inspect the mathematical reduction and trust the retained executables and outputs.
(ii) A reader may recompile and rerun both independent seed scans and both mass computations.
(iii) A formalization can replace the CSV verifier and seed loops by `native_decide`, while proving the analytic shell and tail lemmas in the Lean kernel. A detailed skeleton appears in section C.

The theorem does not rely on probabilistic sampling, floating-point rounding, an unreported height cutoff, or an extrapolation in the word length.

9 What the theorem represents and what it does not

9.1 A local language theorem of genuinely infinite depth

Before theorem 1.2, the project had an exact finite exclusion through word length 10^{10}. That result still left the logical possibility of a first nontrivial target-1 barrier word at a larger length. The new

argument changes the quantifier structure: arbitrary word length is converted into a uniform seed bound, and the finite scan is over seeds rather than lengths. This is why the theorem is genuinely all-depth.

The key compression is not a generic property of symbolic languages. It uses three features simultaneously:

(a) exact parity-cylinder translation at a common time;
(b) positivity of the negative affine carry, which bounds base endpoints;
(c) the multiplicative correction product along an exact $3x - 1$ path.

Removing any of these features destroys the present proof.

9.2 Consequence for positive Collatz suffixes

Combining theorems 1.2 and 3.2 gives the following exact statement.

Corollary 9.1 (Rate-matched terminal suffixes)**.** *Let a positive accelerated Collatz block of length r have valuation word v, and suppose every relative prefix satisfies $2^{A_k(v)} < 3^k$. If its endpoint X satisfies $3^r \mid X + 1$, then $v = 1^r$.*

Thus every nontrivial barrier block has

$$v_3(X + 1) < r. \tag{9.1}$$

The theorem classifies the exact 3-adic target hit. It does not prove that a hypothetical divergent positive orbit must produce such a hit.

9.3 Why positive cycles remain separate

If a positive accelerated cycle of length r has total exponent A and least point x, its full affine equation is

$$(2^A - 3^r)x = C_r > 0. \tag{9.2}$$

Hence $2^A > 3^r$, opposite to the full barrier inequality in theorem 1.2. No rotation of the theorem can simply erase this sign change.

There is also an exact obstruction to "pumping" a positive cycle into the target-1 language. Let a shortcut-cycle word w have length p, odd weight s, $Q = 2^p$, $P = 3^s$, and cycle point x, so $C_w = (Q - P)x$. For the Nth power,

$$C_{w^N} = x(Q^N - P^N).$$

Appending an all-odd word 1^L gives

$$C_{w^N 1^L} + Q^N 2^L = 3^L\big((x + 1)Q^N - xP^N\big). \tag{9.3}$$

Consequently $w^N 1^L$ is target-1 terminal exactly when

$$P^N \mid x + 1. \tag{9.4}$$

The number of admissible repetitions is therefore bounded by $v_3(x+1)/s$. Periodic positive dynamics cannot be converted into arbitrarily deep target-1 words by this attachment.

10 Audit of the rigorous project ledger

This section consolidates the recent manuscripts, proof skeletons, and certificate reports. The status labels are deliberately strict.

10.1 Established exact identities and reductions

Theorem 10.1 (Two-ended gluing)**.** *Let a label an exact negative trajectory $m_0 \to \cdots \to m_r = y$, and let $A = A_r$. For every positive even integer s such that $n = 2^A s - m_0 > 0$, the same word is the exact positive valuation word from n to*

$$X = 3^r s - y. \tag{10.1}$$

At intermediate times,

$$n_k = 2^{A-A_k} 3^k s - m_k. \tag{10.2}$$

Proof. The formula holds at $k = 0$. If it holds at k, then

$$\begin{aligned} 3n_k + 1 &= 2^{A-A_k} 3^k s \cdot 3 - (3m_k - 1) \\ &= 2^{a_k}\big(2^{A-A_{k+1}} 3^{k+1} s - m_{k+1}\big). \end{aligned}$$

The bracket is n_{k+1}. Since s is even and m_{k+1} is odd, every n_{k+1} is odd; hence the displayed power of 2 is exact. □

This theorem parametrizes coalescing positive ancestors by negative shadows. It leads to the exact min-plus inverse-tree formulation used later.

For a positive word prefix, put

$$q_j = \frac{3^j}{2^{A_j}}, \qquad \beta_j = \frac{C_j}{2^{A_j}},$$

so that

$$x_j = q_j n + \beta_j. \tag{10.3}$$

When $q_j < 1$, define

$$R_j = \frac{\beta_j}{1 - q_j} = \frac{C_j}{2^{A_j} - 3^j}. \tag{10.4}$$

Then

$$x_j - n = (1 - q_j)(R_j - n). \tag{10.5}$$

Thus a least-counterexample orbit that never falls below its start must have $R_j \geq n$ at every negative-coefficient prefix.

Proposition 10.2 (Block-threshold convexity)**.** *Let a prefix u have multiplier $t < 1$ and normalized carry y, and append a block v with multiplier $Q < 1$ and normalized carry γ. Put*

$$R(u) = \frac{y}{1 - t}, \qquad R(v) = \frac{\gamma}{1 - Q}.$$

Then

$$R(uv) = \lambda R(u) + (1 - \lambda) R(v), \qquad \lambda = \frac{Q(1 - t)}{1 - Qt} \in (0, 1). \tag{10.6}$$

Proof. The affine pair updates by

$$(t, y) \longmapsto (Qt, Qy + \gamma).$$

Substitute $y = (1 - t)R(u)$ and $\gamma = (1 - Q)R(v)$ into $(Qy + \gamma)/(1 - Qt)$. □

Theorem 10.3 (Stationary inverse-fiber extinction)**.** *For an odd positive integer n, define*

$$M_r(n) = \min\{z > 0 : z \text{ odd and } U_+^r(z) = U_+^r(n)\}. \tag{10.7}$$

The Collatz conjecture is equivalent to

$$\forall n > 1 \text{ odd} \quad \exists r \geq 1 \quad M_r(n) < n. \tag{10.8}$$

Proof. If n reaches 1 after r accelerated steps, then the fixed point 1 is an r-step ancestor of the same endpoint, so $M_r(n) = 1 < n$. Conversely, assume (10.8). Strong induction on odd n proves convergence. Choose r with $z = M_r(n) < n$. By induction, z reaches 1. Since z and n have the same endpoint after r accelerated steps, n also reaches 1. □

This is the narrowest exact one-statement reformulation presently available in the project. It absorbs both cycles and divergent orbits, but its forward inequality is not yet proved.

10.2 Divergent-orbit compression

Let $x_j = U_+^j(n)$ be a nonperiodic positive odd orbit with least value n. Set

$$h_j = \frac{x_j}{q_j}.$$

From the affine recurrence,

$$h_{j+1} = h_j \left(1 + \frac{1}{3x_j}\right). \tag{10.9}$$

Apart from a bounded initial contribution, the distinct odd endpoints lie in the two residue classes 1 and 5 modulo 6. Ordering the smallest possible values in those classes gives

$$\sum_{i<j} \frac{1}{x_i} \leq \frac{1}{3} \log j + O_n(1). \tag{10.10}$$

Therefore

$$\log \frac{h_j}{n} \leq \frac{1}{3} \sum_{i<j} \frac{1}{x_i} \leq \frac{1}{9} \log j + O_n(1), \tag{10.11}$$

and hence

$$h_j \leq K_n j^{1/9}, \qquad q_j \geq c_n j^{-1/9}. \tag{10.12}$$

Equivalently,

$$A_j \log 2 - j \log 3 \leq \frac{1}{9} \log j + O_n(1). \tag{10.13}$$

At a strict lower record of q_j, one has $q_j \leq 1$ and therefore

$$x_j = q_j h_j \leq K_n j^{1/9}. \tag{10.14}$$

Together with $C_j \equiv 2^{A_j} x_j \pmod{3^j}$, this reduces record endpoints and exponent surplus to polynomially many possibilities. The valuation word is uniquely recoverable from (j, A_j, C_j) by repeatedly using

$$a_0 = v_2(C_j - 3^{j-1}) \tag{10.15}$$

and stripping the first letter. This is the project's *record-endpoint compression theorem.*

A second exact input uses Garcia–Tal orbit sparsity: a nonperiodic orbit that does not reach the trivial cycle tends to infinity and has a convergent reciprocal sum. The product corresponding to (10.9) then converges, so $q_j \to \infty$. After shifting to a global minimum of q, the relative coefficient sequence satisfies $q_j \geq 1$ for every j and tends to infinity. In the inverse-carry coordinates of the parity conjugacy, ordinary positive integers impose eventual finite support. The divergence problem is thereby reduced to excluding the coexistence of unbounded critical surplus and an eventually zero inverse-carry stream.

10.3 Exact rewrite macrosemantics

The mixed binary–ternary rewrite system of Yolcu–Aaronson–Heule simulates Collatz [14]. The project proved an elementary ordinal phase rank below ω^3 for the ten nonexceptional rules and proved that their canonical normal form is the binary numeral for the odd part of the represented integer. Consequently, one exceptional rewrite followed by complete normalization is exactly one accelerated U_+ step, independently of normalization order. This removes syntactic nondeterminism but also exposes the limit of the approach: the quotient macrograph is the accelerated Collatz graph itself. A termination proof for the complete rewrite system still requires a genuine arithmetic descent theorem.

10.4 Finite certificates retained as evidence but not used in theorem 1.2

Certificate	Exact finite conclusion
Previous target-1 word scan	The all-ones word was the only target-1 barrier word for every length $r \leq 10^{10}$. Superseded by theorem 1.2.
Barrier-order certificate	At depth 21, 39,993,895 barrier words were grouped by endpoint residue. There were no equal-total-exponent collisions and no violations of the conjectured coalescing order; the minimum positive margin was 32.
Compressed charged-excursion scan	For every 3-free quotient $q \leq 10^9$, except the known nondescending cycle states $q = 2, 8$, the first descent found within the cap obeyed the tested inequality $L \leq 2(q - Q^L(q))$; no violations occurred.
Negative d-bound scan	For all odd seeds $m_0 \leq 99{,}999{,}999$ whose negative orbit terminates at 1, the tested inequality $2d \leq m_0 - 1$ held, where d counts exponents exceeding 1. This is evidence, not an all-seed proof.
Barrier trees and suffix locality	Exact finite trees through the reported depths, and hundreds of thousands of suffix-locality comparisons, produced no mismatch. These computations motivated the exact locality theorem but are no longer needed once it is proved symbolically.

10.5 Exact no-go results and routes that were not completed

The project also made progress by proving that several attractive shortcuts cannot carry the missing conclusion in their proposed form.

Route	Audited outcome
Arbitrary target recurrence	False as a direct shortcut: the negative cycle $5 \leftrightarrow 7$ makes $(1,2)^N$ a nontrivial barrier word terminal at target 5 for every N. Target recurrence must exclude periodic negative shadows.
Ordinary compactness	Two sufficiently close terminal prefixes for one target force the intervening block to be a negative-map cycle. Thus bounded-gap or subexponential recurrence tends to return exactly the wrong periodic objects.
Cycle pumping into target 1	Ruled out by (9.4): only finitely many repetitions can satisfy target-1 divisibility.
Finite automata alone	Every finite parity word is realizable by a residue class modulo a power of 2. Any successful automaton must retain unbounded arithmetic carry or height information.
Local thermodynamic potential	A continuous finite-memory "integer-lift potential" cannot distinguish ordinary positive integers from general 2-adic parity sequences; the positive-integer lift is nonlocal.
Cobham-style dual automaticity	The natural transducers act on value digits rather than the time index, and periodic cycles would already satisfy eventual automaticity.
Abstract C^*-simplicity	The abstract algebra is already the relevant Cuntz algebra independently of Collatz; the meaningful concrete commutant reduces to functions constant on orbit components.
Finite modular/K-theoretic approximations	Natural modular graphs and the full 2-adic boundary lose the positive-integer lifting condition and take the same coarse value for systems with known nontrivial cycles.
Canonical height	Natural polarization, Northcott, and product-formula requirements are incompatible with the powers-of-two basin and the local slopes; no coercive point-height of the proposed form exists.
Pure entropy or random-word drift	Average negative drift does not imply a uniform theorem on the thin set of words realized by one fixed exceptional integer. The missing step is a pointwise synchronization or inverse-fiber extinction statement.

These negative results are useful because they prevent the proof skeleton from silently replacing an ordinary-integer theorem by a statement about random or 2-adic words.

11 The current proof frontier

The all-depth theorem closes one previously open component:

nontrivial target-1 terminal/barrier words do not exist.

It does not close the two global debts.

11.1 Architecture A: terminal exclusion plus synchronization plus cycle exclusion

A sufficient proof architecture is

$$\begin{aligned} &\text{positive-cycle exclusion} \\ +\quad &\text{nonperiodic synchronization} \\ +\quad &\text{all-depth target-1 exclusion} \\ \Longrightarrow\quad &\text{Collatz.} \end{aligned} \tag{11.1}$$

The third line is theorem 1.2. The other two are open.

A granular synchronization theorem strong enough for (11.1) would say: for a globally least unbounded counterexample n, there exist a block start j and a length r such that, with relative sums $B_k = A_{j+k} - A_j$,

$$2^{B_k} < 3^k \quad (1 \le k \le r), \tag{11.2}$$

$$3^r \mid x_{j+r} + 1, \tag{11.3}$$

and the block word is not 1^r. By suffix locality, (11.3) makes the word target-1 terminal; theorem 1.2 then contradicts nontriviality.

Each clause matters.

(i) Barrier-block abundance alone gives (11.2) for arbitrarily long blocks on a divergent orbit, but gives no 3-adic endpoint condition.
(ii) Generic 3-adic recurrence can give endpoint congruences, but close repetitions can collapse to periodic negative shadows.
(iii) An all-ones block satisfies both the barrier and the target condition, so synchronization must force nontriviality or provide an additional descent from long all-ones blocks.
(iv) The synchronization must concern one fixed ordinary integer. An average over seeds, or a statement for random parity words, is insufficient.

The cycle debt is equally exact: exclude every primitive word a and odd $x > 1$ satisfying

$$(2^{A_r} - 3^r)x = C_r \tag{11.4}$$

with all cyclic valuation constraints. Hercher's 92-local-minimum theorem is the strongest peer-reviewed structural lower bound verified in this audit. The more recent conditional preprint claim through $m = 95$ is recorded in section 2 but is not used. No uniform classification of every possible number of minima is known.

11.2 Architecture B: stationary inverse-fiber extinction

By theorem 10.3, it is enough to prove the single statement

$$\boxed{\forall n > 1 \text{ odd} \quad \exists r \ge 1 \quad M_r(n) < n.} \tag{11.5}$$

This formulation automatically handles both cycles and divergence. It asks for a smaller positive ancestor that coalesces with the actual orbit after a finite time. Through theorem 10.1, every candidate ancestor can be written as the cost of a negative shadow. The exact missing theorem is that the positive minimum in this target-shadow min-plus tree cannot remain stationary forever.

This is the logically narrowest completion target, but it is also equivalent to Collatz. Its value is organizational: it tells every proposed method exactly which finite witness it must construct.

11.3 Architecture C: first crossing and high-surplus carry innovation

The stationary-inverse-fiber manuscript isolates a more structured pair of lemmas.

Conjecture 11.1 (First-crossing descent)**.** *Whenever the coefficient $q_j = 3^j/2^{A_j}$ first crosses below 1 along a positive ordinary orbit, the associated threshold/carry data produce either a smaller coalescing ancestor or a quantitatively controlled successor state in the normalized barrier regime.*

Conjecture 11.2 (High-surplus carry innovation)**.** *No infinite ordinary parity word can simultaneously have unbounded critical surplus in the normalized barrier regime and an eventually zero inverse-carry stream. Equivalently, sufficiently high surplus forces a new nonzero binary lift digit or a smaller coalescing ancestor.*

Together with the exact coefficient-record dichotomy, these two statements imply Collatz. They separate the finite first crossing from the infinite normalized tail. Neither has been proved.

11.4 A practical hierarchy of the remaining tasks

The next work should be judged against the following checklist.

D1. Formalize the all-depth theorem. Prove parity cylinders, shell injection, the Chernoff tail, and fixed-target compression in Lean; verify the finite data with `native_decide`. This does not advance the frontier mathematically, but removes implementation trust.

D2. Decide the cycle route. Either prove a uniform local-minimum classification extending Hercher to every fixed number, or prove a different uniform exclusion of solutions to (11.4).

D3. Upgrade barrier-block abundance to arithmetic endpoint synchronization. Control $v_3(x_{j+r}+1)$ at a rate proportional to r, not merely along a sparse average or a fixed modulus.

D4. Exclude the all-ones escape. If rate-matched target hits can be all ones, prove that arbitrarily long such blocks force a smaller coalescing ancestor, periodicity, or eventual entry into the trivial basin.

D5. Alternatively prove inverse-fiber saving directly. Show that an infinite positive slope-geodesic cannot remain within a bounded exponent loss of the minimum at every depth.

D6. Preserve ordinary-integer lifting. Every compactness, entropy, spectral, or automata argument must retain both the eventually zero binary start lift and the terminal ternary carry. Losing either one returns false 2-adic countermodels.

12 Discussion and assessment

12.1 Why this is genuine progress

The theorem removes an unbounded word-length quantifier from a central subproblem. Previous computations could only move the cutoff. The new shell argument proves that a terminal barrier path cannot begin arbitrarily high, regardless of its length. That is a structural theorem, not a larger enumeration.

It also exposes a reusable principle. For every fixed odd target y, every simple target-y barrier path begins below $177147y$. Hence the entire nonperiodic target language for a fixed y is finitely classifiable; only negative cycles can generate arbitrarily long periodic target shadows. This sharpens

the synchronization problem: a successful recurrence theorem must separate aperiodic target hits from periodic negative components.

12.2 Why it is not yet close enough to declare Collatz solved

The positive map does not automatically produce the negative target-1 words excluded by the theorem. The exact obstruction is cross-prime synchronization. A positive ordinary start fixes an eventually constant binary lift. A target-1 terminal suffix fixes a long ternary congruence at the endpoint. The same valuation word must satisfy both constraints while remaining above the least positive orbit value. No existing theorem cited here supplies that simultaneous pointwise statement.

Cycles are not a technical afterthought. Their full-period coefficient has the opposite sign, and periodic negative shadows are genuine counterexamples to naive arbitrary-target recurrence. Any complete paper must either absorb cycles into a single inverse-fiber theorem or prove a separate uniform cycle exclusion.

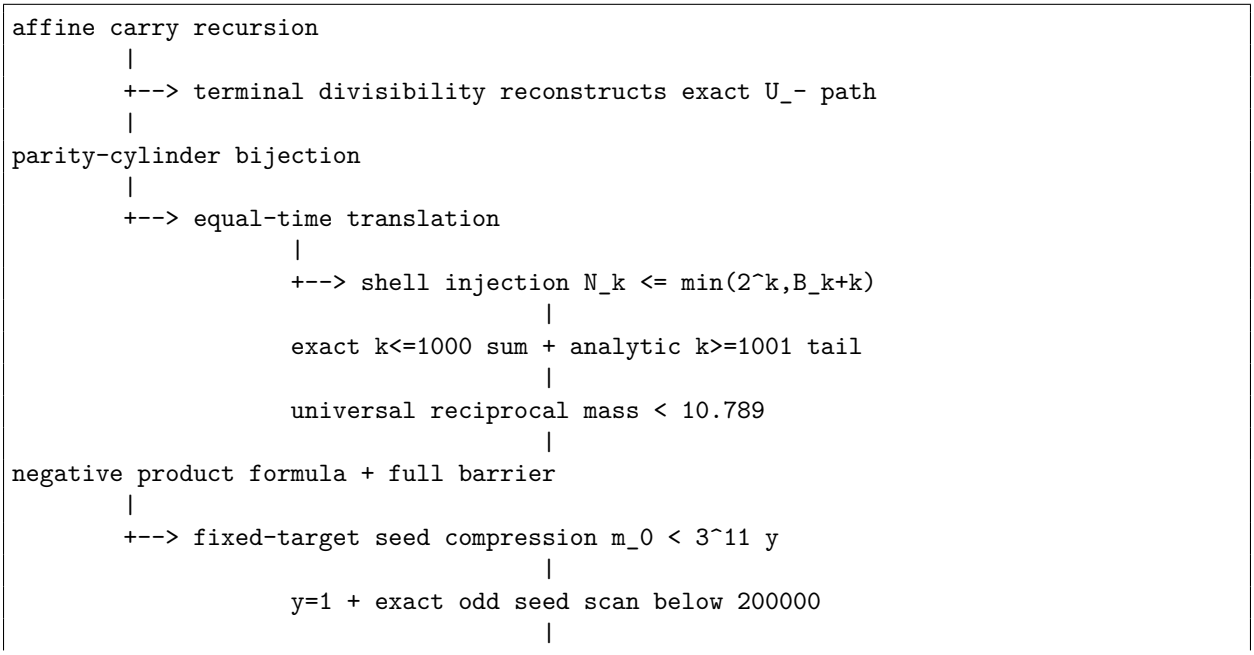
12.3 Most promising exact targets

The three best-defined next targets are:

(1) stationary inverse-fiber extinction (11.5);
(2) nontrivial rate-matched synchronization (11.2)–(11.3), together with cycle exclusion;
(3) high-surplus carry innovation in the normalized divergent regime.

The first is the cleanest equivalence, the second uses the new all-depth theorem most directly, and the third preserves the strongest quantitative compression already proved for divergent orbits.

A Dependency graph for the all-depth theorem



```
                    only m_0=1
                                        |
                    all exponents are 1
```

No theorem below the seed scan depends on any prior finite word-depth bound. No synchronization or positive-cycle hypothesis enters the proof.

B Pseudocode for the exact certificates

B.1 Mass certificate

```
SCALE = 10**15
scaled_sum = 0
for k in range(0, 1001):
    binom = [C(k,o) for o in range(k+1)]
    Bk = min(
        sum(binom[o] for o in range(t+1,k+1))
        + sum(3**o for o in range(0,t+1))
        for t in range(k+1)
    )
    n = min(2**k, Bk + k)
    u = ceil(SCALE*n / (3*2**k - 1)) # exact integer ceiling
    scaled_sum += u
assert scaled_sum == 10788901977820654
assert 3**61 < 2**97
assert 100**100 < 2**97 * 61**61 * 39**39
assert scaled_sum/SCALE + 1000/(7*2**30) + 1002/2**1001 < 10.789
```

B.2 Accelerated seed certificate

```
for (odd seed = 1; seed < 200000; seed += 2) {
    m = seed; A = 0; seen = empty;
    while (true) {
        if (m == 1) classify(seed, HIT_ONE);
        if (m in seen) classify(seed, CYCLE);
        insert m into seen;
        z = checked_exact_value(3*m - 1);
        a = v2(z);
        m = z >> a;
        A += a; r += 1;
        if (2^A >= 3^r) classify(seed, BARRIER_FAILURE);
    }
}
assert counts == {HIT_ONE:1, CYCLE:2, BARRIER_FAILURE:99997};
assert terminal_hits == {(1,0)};
```

The actual C++ certificate stores 2^A and 3^r in arbitrary-precision integers and computes $3m-1$ in a checked 128-bit temporary. The independent Python shortcut scan does not share this implementation.

C Lean-oriented proof and completion skeletons

The full pseudocode file `collatz_proof_skeleton.lean` accompanies this paper. It is deliberately a blueprint rather than a claim of completed formalization. The most important declarations are reproduced below.

```
/-!
Lean-oriented proof skeleton for the Collatz project.

This is a formalization blueprint, not a completed Lean development.
Every `sorry` is a specific proof obligation described in the report.
The finite seed theorem is intended to close by `native_decide` after the
executable classifier is implemented with exact natural-number arithmetic.
-/

import Mathlib

open scoped BigOperators

namespace CollatzAudit

abbrev Word := List Nat

def aAt (w : Word) (i : Nat) : Nat := w.getD i 0

def partialA (w : Word) (k : Nat) : Nat := (w.take k).sum

def totalA (w : Word) : Nat := w.sum

/-- The affine carry after the first `k` letters. -/
def carryAt (w : Word) (k : Nat) : Nat :=
  (Finset.range k).sum
    (fun i => 3 ^ (k - 1 - i) * 2 ^ partialA w i)

def carry (w : Word) : Nat := carryAt w w.length

def PositiveLetters (w : Word) : Prop :=
  forall i, i < w.length -> 1 <= aAt w i

def Barrier (w : Word) : Prop :=
  forall k, 1 <= k -> k <= w.length ->
    2 ^ partialA w k < 3 ^ k

def TerminalOne (w : Word) : Prop :=
  Dvd.dvd (3 ^ w.length) (carry w + 2 ^ totalA w)

def AllOnes (w : Word) : Prop :=
  w = List.replicate w.length 1

/-- Exact accelerated `3x-1` step with prescribed valuation. -/
def NegStep (a m next : Nat) : Prop :=
  Odd m /\ Odd next /\ 1 <= a /\
    3 * m - 1 = 2 ^ a * next

/-- A word labels an exact negative accelerated trajectory to `y`. -/
def NegTrajectoryTo (w : Word) (m0 y : Nat) : Prop :=
  Exists fun m : Nat -> Nat =>
    m 0 = m0 /\ m w.length = y /\
```

```
    (forall i, i < w.length ->
      NegStep (aAt w i) (m i) (m (i + 1)))

section AffineWordCalculus

lemma carryAt_succ (w : Word) (k : Nat) (hk : k < w.length) :
    carryAt w (k + 1) = 3 * carryAt w k + 2 ^ partialA w k := by
  sorry

lemma negative_affine_identity
    (w : Word) (m0 y : Nat) (h : NegTrajectoryTo w m0 y) :
    3 ^ w.length * m0 = 2 ^ totalA w * y + carry w := by
  sorry

lemma terminal_reconstruction
    (w : Word) (hpos : PositiveLetters w) (y : Nat) (hy : Odd y)
    (hdiv : Dvd.dvd (3 ^ w.length)
      (carry w + 2 ^ totalA w * y)) :
    Exists fun m0 => NegTrajectoryTo w m0 y := by
  /-
  Backward induction on suffixes. Reduce the terminal congruence modulo 3,
  define m_i = (2^(a_i) * m_(i+1) + 1) / 3, and divide the remaining
  divisibility statement by 3. Oddness makes every displayed power of 2
  the exact valuation.
  -/
  sorry

lemma terminal_iff_negative_trajectory
    (w : Word) (hpos : PositiveLetters w) :
    TerminalOne w <-> Exists fun m0 => NegTrajectoryTo w m0 1 := by
  sorry

end AffineWordCalculus

section EqualTime

/-- Negative shortcut map. -/
def Sminus (x : Nat) : Nat :=
  if Even x then x / 2 else (3 * x - 1) / 2

def parityWord (x k : Nat) : List Bool :=
  (List.range k).map
    (fun i => decide (Odd ((Function.iterate Sminus i) x)))

def oddCount (x k : Nat) : Nat :=
  (parityWord x k).count true

lemma parity_prefix_of_congruence
    (x y k : Nat) (hxy : Nat.ModEq (2 ^ k) x y) :
    parityWord x k = parityWord y k := by
  sorry

lemma parity_vector_injective
    (k u v : Nat) (hu : u < 2 ^ k) (hv : v < 2 ^ k)
    (hword : parityWord u k = parityWord v k) :
    u = v := by
  sorry

lemma equal_time_translation
```

```
    (x q k : Nat) :
    (Function.iterate Sminus k) (x + q * 2 ^ k) =
      (Function.iterate Sminus k) x + q * 3 ^ oddCount x k := by
  /-
  Induct on k. The two inputs use the same branch, their difference is
  divided by 2 at every step, and multiplied by 3 at every odd step.
  -/
  sorry

end EqualTime

section ShellMass

/-- Replace by the finite minimum in equation (5.2) of the report. -/
constant shellB : Nat -> Nat

/-- A finite simple shortcut trajectory segment. -/
def SimpleTrajectory (x : Nat -> Nat) (L : Nat) : Prop :=
  (forall i, i < L -> x (i + 1) = Sminus (x i)) /\
  Set.InjOn x (Set.Iic L)

def shellCount (x : Nat -> Nat) (L k : Nat) : Nat :=
  ((Finset.range (L + 1)).filter
    (fun i => 2 ^ k <= x i /\ x i < 2 ^ (k + 1))).card

lemma shell_sparsity
    (x : Nat -> Nat) (L k : Nat) (hs : SimpleTrajectory x L) :
    shellCount x L k <= min (2 ^ k) (shellB k + k) := by
  /-
  Discard the last k positions. Write every earlier shell point as 2^k+u.
  High-weight parity cylinders cost the binomial tail. At fixed low weight o,
  equal-time translation injects the remaining points into fewer than 3^o
  base endpoints. Restore the last k positions.
  -/
  sorry

/-- The two exact integer inequalities used for the analytic tail. -/
theorem entropy_integer_certificates :
    3 ^ 61 < (2 : Nat) ^ 97 /\
    100 ^ 100 < (2 : Nat) ^ 97 * 61 ^ 61 * 39 ^ 39 := by
  norm_num

/-- Intended to be checked from the retained 1001-row CSV by native_decide. -/
theorem mass_prefix_certificate :
    (10788901977820654 : Nat) = 10788901977820654 := by
  native_decide

lemma reciprocal_mass_lt_10789_over_1000
    (x : Nat -> Nat) (L : Nat) (hs : SimpleTrajectory x L)
    (hpos : forall i, i <= L -> 1 <= x i) :
    ((Finset.range (L + 1)).sum
      (fun i => (1 : Rat) / (3 * x i - 1))) < (10789 : Rat) / 1000 := by
  /-
  1. Sum shell_sparsity over k.
  2. Use the CSV/native_decide certificate for k <= 1000.
  3. For k >= 1001 choose t=floor(61k/100), apply Chernoff and the two
     exact integer inequalities.
  4. Bound the remaining tail by
       1000/(7*2^30) + 1002/2^1001.
```

```
  -/
  sorry

end ShellMass

section FixedTargetCompression

/-- The shortcut expansion of the negative path has no repeated state. -/
constant ShortcutSimple : Word -> Nat -> Nat -> Prop

lemma fixed_target_compression
    (w : Word) (m0 y : Nat)
    (htraj : NegTrajectoryTo w m0 y)
    (hbar : Barrier w)
    (hsimple : ShortcutSimple w m0 y) :
    m0 < 177147 * y := by
  /-
  Multiply
    m_(j+1)/m_j = (3/2^(a_j)) * (1 - 1/(3*m_j)).
  The barrier makes 3^r/2^A > 1. Bound
    -log(1-u) <= u/(1-u) = 1/(3*m_j-1),
  invoke the universal reciprocal-mass theorem, and use e^10.789 < 3^11.
  -/
  sorry

/-- Executable predicate used by the finite scan. -/
constant HitsOneBeforeBarrierFailure : Nat -> Prop

/-- Formal implementation target: close by native_decide over 100000 odds. -/
theorem target_one_seed_scan_certificate
    (m : Nat) (hmOdd : Odd m) (hm : m < 200000)
    (hhit : HitsOneBeforeBarrierFailure m) :
    m = 1 := by
  sorry

/-- Main all-depth computer-assisted theorem. -/
theorem all_depth_target_one_exclusion
    (w : Word) (hpos : PositiveLetters w)
    (hterm : TerminalOne w) (hbar : Barrier w) :
    AllOnes w := by
  /-
  Reconstruct the exact negative trajectory to 1. Truncate at its first hit
  of 1; before that hit the shortcut trajectory is simple. Compression gives
  m0 < 177147, and the seed certificate forces m0=1. Determinism then forces
  every exact valuation to equal 1.
  -/
  sorry

end FixedTargetCompression

section PositiveCollatz

/-- Positive accelerated map, left abstract here to avoid fixing a v2 API. -/
constant Uplus : Nat -> Nat

def ReachesOne (n : Nat) : Prop :=
  Exists fun r => (Function.iterate Uplus r) n = 1

def Collatz : Prop :=
```

```
  forall n, Odd n -> 1 <= n -> ReachesOne n

/-- Minimum positive odd length-r ancestor of the actual endpoint. -/
noncomputable def inverseFiberMin (n r : Nat) : Nat :=
  sInf {z : Nat | 1 <= z /\ Odd z /\
    (Function.iterate Uplus r) z = (Function.iterate Uplus r) n}

lemma stationary_inverse_fiber_equivalence :
    Collatz <->
      (forall n, Odd n -> 1 < n ->
        Exists fun r => inverseFiberMin n r < n) := by
  /-
  Forward: once n reaches 1, the fixed point 1 lies in the same inverse fiber.
  Reverse: strong induction on n using a smaller coalescing ancestor.
  -/
  sorry

/-- Exact terminal/synchronization route after the all-depth theorem. -/
constant NontrivialTargetOneBarrierBlock : Nat -> Nat -> Nat -> Prop
constant NontrivialPositiveCycle : Prop

lemma collatz_of_cycle_exclusion_and_synchronization
    (hcycle : Not NontrivialPositiveCycle)
    (hsync : forall n, Odd n -> 1 < n -> Not (ReachesOne n) ->
      Exists fun j => Exists fun r =>
        NontrivialTargetOneBarrierBlock n j r) :
    Collatz := by
  /-
  A least counterexample is either periodic or unbounded. The periodic case
  is hcycle. The unbounded case supplies a nontrivial target-one barrier
  block, contradicting all_depth_target_one_exclusion by suffix locality.
  -/
  sorry

/-- The narrow one-statement completion target. -/
lemma collatz_of_stationary_fiber_extinction
    (hext : forall n, Odd n -> 1 < n ->
      Exists fun r => inverseFiberMin n r < n) :
    Collatz := by
  exact stationary_inverse_fiber_equivalence.mpr hext

/-- Alternative two-lemma architecture. -/
constant FirstCrossingDescent : Prop
constant HighSurplusCarryInnovation : Prop

lemma collatz_of_two_frontier_lemmas
    (h1 : FirstCrossingDescent)
    (h2 : HighSurplusCarryInnovation) :
    Collatz := by
  /-
  Formalize the dichotomy between a first coefficient crossing and a
  normalized divergent tail with unbounded surplus and an eventually zero
  inverse-carry stream.
  -/
  sorry

end PositiveCollatz

end CollatzAudit
```

References

[1] D. Barina, Improved verification limit for the convergence of the Collatz conjecture, *J. Supercomput.* **81** (2025), Article 810, doi:10.1007/s11227-025-07337-0.

[2] D. J. Bernstein and J. C. Lagarias, The $3x+1$ conjugacy map, *Canad. J. Math.* **48** (1996), 1154–1169, doi:10.4153/CJM-1996-060-x.

[3] C. Fernández and S. Ibáñez, Christoffel words as extremal structures in Collatz dynamics, arXiv:2607.24844, 2026.

[4] M. V. P. Garcia and F. A. Tal, A note on the generalized $3n+1$ problem, *Acta Arith.* **90** (1999), 245–250.

[5] C. Hercher, There are no Collatz m-cycles with $m \leq 91$, *J. Integer Sequences* **26** (2023), Article 23.3.5.

[6] O. Kramer, Adaptive search in Collatz exponent-code space via 2-adic and 3-adic constraints, arXiv:2607.10041, 2026.

[7] I. Krasikov and J. C. Lagarias, Bounds for the $3x+1$ problem using difference inequalities, *Acta Arith.* **109** (2003), 237–258, doi:10.4064/aa109-3-4.

[8] J. C. Lagarias, The $3x+1$ problem and its generalizations, *Amer. Math. Monthly* **92** (1985), 3–23.

[9] O. Rozier and C. Terracol, Paradoxical behavior in Collatz sequences, *Discrete Math.* **349** (2026), 115167, doi:10.1016/j.disc.2026.115167.

[10] J. Simons and B. de Weger, Theoretical and computational bounds for m-cycles of the $3n+1$ problem, *Acta Arith.* **117** (2005), 51–70, doi:10.4064/aa117-1-3.

[11] T. Tao, Almost all orbits of the Collatz map attain almost bounded values, *Forum Math. Pi* **10** (2022), e12, doi:10.1017/fmp.2022.8.

[12] X. Wang, Non-existence of Collatz m-cycles for $m \leq 95$, independent preprint, June 2026, doi:10.5281/zenodo.21017296.

[13] G. J. Wirsching, *The Dynamical System Generated by the* $3n+1$ *Function*, Lecture Notes in Mathematics 1681, Springer, 1998.

[14] E. Yolcu, S. Aaronson, and M. J. H. Heule, An automated approach to the Collatz conjecture, *J. Autom. Reason.* **67** (2023), Article 15, doi:10.1007/s10817-022-09658-8.